Date:27 October 2026

Cairo university

Faculty of computing and AI

CS213 - Object-Oriented Programming (Assignment 2 - Part 1)

Vole Machine Simulator Report Template

Course: CS213 Object-Oriented Programming
Assignment: Assignment 2 - Part 1
Section: S33 / S34

Course Instructor: Dr. Mohammad El-Ramly

Team Members

Member Name	Student ID	Role / Contribution
Mariam Tarig Omer (leader)	20253009 (phone num:01500343024)	UML Design & AI Comparison
Shahd Abdalelah Osman	20253012	Video Explanation & Documentation
Mozan Nageeb Mohamed	20253016	GitHub Setup & PlantUML Diagrams
Rana Salah Aldeen Khalid	20253018	Program Testing & Report Drafting

- **Team members email's:**
- 20253009@stud.fci-cu.edu.eg
- 20253012@stud.fci-cu.edu.eg
- 20253018@stud.fci-cu.edu.eg
- 20253016@stud.fci-cu.edu.eg

1. Introduction And Vole Architecture Overview

In Part 1 of this project we analyzed the hardware architecture of the Vole Machine to design our C++ simulator. The architecture operates on a 256-cell Main Memory (addresses 00 through FF) where

address FF handles memorymapped screen output. The processor utilizes 16 general-purpose registers (R0 through RF) an 8-bit Program Counter (PC) to store the next instruction address, and a 16-bit Instruction Register (IR) to hold the current instruction being executed.

2. Initial Design :

Before consulting AI tools, our team proposed a baseline Object-Oriented architecture for the Vole Machine Simulator based on the system hardware requirements. The architecture consists of 5 core classes:

1. **Memory:** Manages the 256 memory bytes (addresses 00 to FF), handling raw byte reading and writing.
2. **RegisterFile:** Manages the 16 general-purpose registers (R0 through RF).
3. **Instruction:** Represents and decodes 16-bit hex instructions into opcodes and operands.
4. **CPU:** Contains the Program Counter (PC) and Instruction Register (IR), managing the Fetch-Decode-Execute cycle (step-by-step or continuous execution).
5. **VoleSimulator (Machine):** The main controller class that owns the Memory, RegisterFile, and CPU, coordinating overall system simulation.

PlantUML Code:

```
@startuml

skinparam class AttributeIconSize 0

skinparam linetype ortho



class Memory {

  - cells: std::array<uint8_t, 256>

  + read(address: uint8_t): uint8_t

  + write(address: uint8_t, value: uint8_t): void

}



class RegisterFile {

  - registers: std::array<uint8_t, 16>

  + get(index: uint8_t): uint8_t

  + set(index: uint8_t, value: uint8_t): void
```

```
}

class Instruction {
  - raw: uint16_t
  + opcode(): uint8_t
  + operand1(): uint8_t
  + operand2(): uint8_t
}

class CPU {
  - pc: uint8_t
  - ir: uint16_t
  + fetch(mem: Memory&): void
  + execute(regs: RegisterFile&): void
}

class VoleSimulator {
  - memory: Memory
  - registers: RegisterFile
  - cpu: CPU
  + step(): void
  + run(): void
}

VoleSimulator *-- Memory
```

VoleSimulator *-- RegisterFile

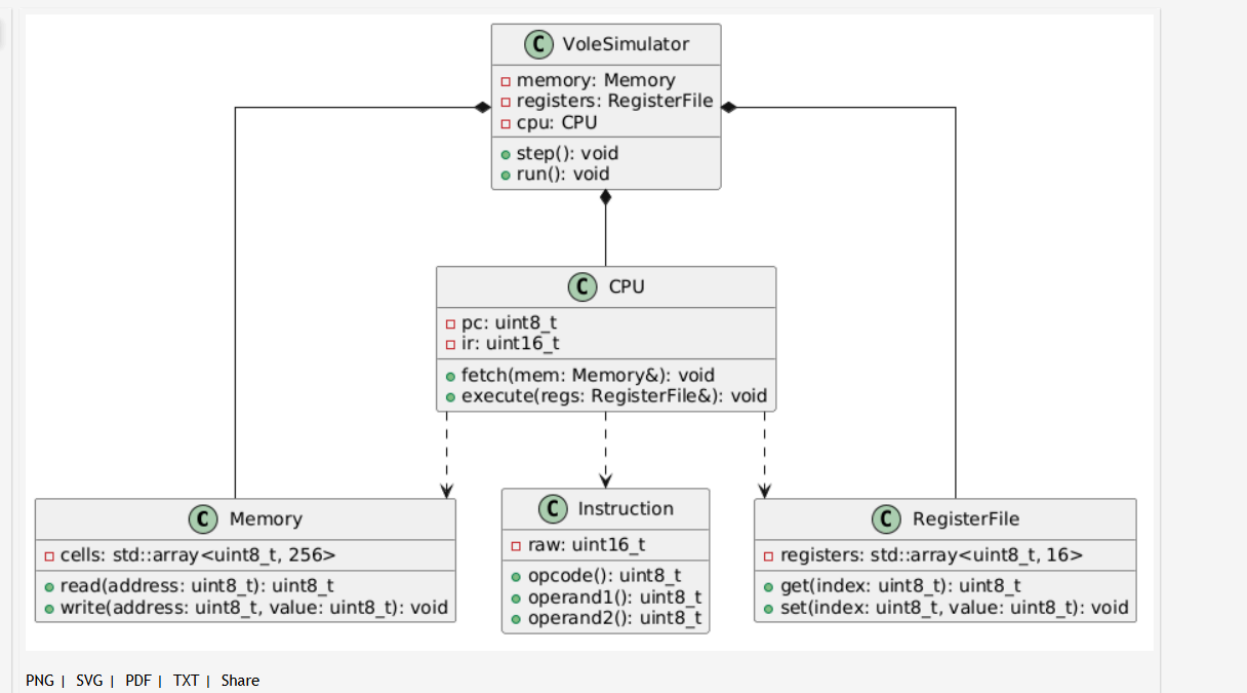
VoleSimulator *-- CPU

CPU ..> Memory

CPU ..> RegisterFile

CPU ..> Instruction

@enduml



3.AI Designs :

Prompts Used:

Chat link:https://claude.ai/share/587a1e5a-75a4-40e8-9b23-aed1eb41771c

First AI Prompt :

احنا شغالين على تصميم محاكي لآلة vole machine بلغة c++ باستخدام مفهوم البرمجة الكائنية oop. الآلة بتتكون من ذاكرة main memory فيها 256 مكان من 00 لحد ff، مسجل 16 و register من r0 لـ rf، معالج cpu فيه عداد البرنامج pc ومسجل التعليمة ir، التصميم المبدئي المقترح منا فيه run. أو مرة واحدة step by step بالإضافة لكلاس رئيسي للمحاكي بيمشي الأوامر إما خطوة بخطوة عشان يفك تشفير الأمر instruction عشان ندير المسجلات الـ 16 register عشان نحفظ ونقرا العناوين memory: الكلاسات دي وده الكلاس الأساسي اللي simulator أو machine عشان يجيب الأوامر ويفهمها وينفذها cpu المكون من 4 خانات هكس ويتأكد منه بيجمع السيستم كله

للرسمة مع plantuml المطلوب منك تراجع التصميم ده وتقترح أي تحسينات على الكلاسات والخصائص والعلاقات بينها واكتب لي كود شرح بسيط ليه اخترت العلاقات دي بين الكلاسات

why you did the design this way??

REPLIES:

مراجعة التصميم المبدئي #

التصميم اللي اقترحتوه سليم في الأساس، وفصل المسؤوليات واضح. عندي كذا ملاحظة تحسّن الجودة:

1. الاتنين عبارة عن "مصفوفة بخانات بتتقرا وتتكتب"، فممكن نعمل لهم كلاس أب **Register: تكرار منطق الـ Memory والـ مشترك أو على الأقل نوحّد الواجهة.

2. الـ Instruction فيه الـ (immutable) ممكن يكون أقوى: بدل ما هو بس "يفك التشفير"، نخليه كائن غير قابل للتعديل والمعاملات (R, X, Y, S, XY) ودالة `isValid()`. وبعدين ننفذ كل أمر بنمط **Command/Strategy** بدل `switch` ضخم جوه الـ CPU، فيبقى إضافة أمر جديد من غير تعديل الـ CPU (مبدأ Open/Closed).

3. فصل الـ I/O: أمر 3 (store في 00) والشاشة محتاجين كلاس منفصل زي `Screen`/`IOHandler`.

4. الـ IR و الـ PC: لازم يكونوا جوه الـ CPU (composition) زي ما قلتم، بس الـ PC هنخليه رقم 8 بت، والـ IR كائن Instruction.

5. الـ Machine: هو اللي يملك الـ CPU والـ Registers والـ Memory ويحمّل البرنامج من ملف (`loadProgram`)، ويوفر `displayState()`، وكمان `reset()` و `run()` و `step()`.

6. حالة التوقف: نضيف `halted` flag في الـ CPU (أمر C000).

ليه اخترت العلاقات دي؟ ##`

| العلاقة | السبب |

|---|---|

| **Machine ◆— Memory / RegisterFile / CPU / Screen** (Composition) | هو اللي بيخلق الأجزاء الـ Machine دي وبيتحكم في عمرها؛ لو الـ Machine اتمسح كله بيتمسح معاه. |

| **CPU ◆— Instruction** (Composition) | جزء داخلي من الـ CPU الـ IR ومش بيعيش لوحده. |

| **CPU ⇢ Memory / RegisterFile** (Dependency) | الـ CPU مش بيملكهم، بيستخدمهم بس وقت الـ cycle (بيجيلها قابل للاختبار لوحده CPU كمعاملات). ده بيخلي الـ (unit testing). |

| **Command ◁— الأوامر الفرعية** (Inheritance) | كل أمر (LOAD, STORE, ADD...) ليه تنفيذ مختلف لنفس الواجهة `execute()` (Polymorphism)، وبكده بنتخلص من `switch` الكبير. |

| **CommandFactory ⇢ Command** | بتفصل مسؤولية "اختيار الأمر المناسب من الـ opcode" عن الـ CPU. |

| **ExecutionContext o— …** (Aggregation) | مجرد حزمة مراجع لحاجات مملوكة لغيره، بتتمرر للأوامر عشان ما نعدّيش 5 معاملات في كل `execute`. |

ملحوظات تنفيذ سريعة ##

- استخدم `uint8_t` بلاش `int` عشان الـ overflow يشتغل زي العتاد الحقيقي للذاكرة والمسجلات.

- وتشيل الكلاسات `CPU::execute()` جوه `switch` زيادة على حجم المشروع، تقدر تبسّطها بـ Command/Factory لو شايف الـ Bitwise) دي من الرسمة، لكن التصميم ده أنضف وأسهل في التوسعة (مثلاً أوامر جديدة زي.

- أمر الـ Float (5 في Vole 8 ممكن تحطها كـ `static` في كلاس `FloatUtils`.

لو حابب أكتب لك هيكل الكود C++ (الـ headers) للتصميم ده أو أوضح الـ sequence diagram لدورة fetch-decode-execute، قولي.

PlantUML CODE:

```
@startuml VoleMachine

skinparam classAttributeIconSize 0

skinparam linetype ortho


class Memory {

  - cells : array<uint8_t, 256>

  + read(address : uint8_t) : uint8_t

  + write(address : uint8_t, value : uint8_t) : void

  + clear() : void

  + loadProgram(program : vector<uint8_t>, start : uint8_t) : void

  + display() : void

}


class RegisterFile {

  - regs : array<uint8_t, 16>

  + get(index : uint8_t) : uint8_t

  + set(index : uint8_t, value : uint8_t) : void

  + clear() : void

  + display() : void

}


class Instruction {

  - raw : uint16_t

  + Instruction(raw : uint16_t)
```

```
  + opcode() : uint8_t
  + regR() : uint8_t
  + regX() : uint8_t
  + regY() : uint8_t
  + regS() : uint8_t
  + address() : uint8_t
  + isValid() : bool
  + toString() : string
}

abstract class Command {
  + {abstract} execute(ctx : ExecutionContext&) : void
}

class LoadFromMemory
class LoadImmediate
class StoreToMemory
class MoveRegister
class AddTwosComplement
class AddFloat
class Jump
class Halt

class ExecutionContext <<struct>> {
```

```
  + memory : Memory&
  + registers : RegisterFile&
  + pc : uint8_t&
  + halted : bool&
  + screen : Screen&
}

class CommandFactory {
  + {static} create(opcode : uint8_t) : unique_ptr<Command>
}

class Screen {
  + print(value : uint8_t) : void
}

class CPU {
  - pc : uint8_t
  - ir : Instruction
  - halted : bool
  + fetch(memory : Memory&) : void
  + decode() : unique_ptr<Command>
  + execute(cmd : Command&, ctx : ExecutionContext&) : void
  + cycle(memory : Memory&, regs : RegisterFile&, screen : Screen&) : void
  + isHalted() : bool
```

```
  + getPC() : uint8_t
  + reset() : void
}

class Machine {
  - memory : Memory
  - registers : RegisterFile
  - cpu : CPU
  - screen : Screen
  + loadProgram(filename : string) : bool
  + step() : void
  + run() : void
  + reset() : void
  + displayState() : void
}

Machine *-- Memory
Machine *-- RegisterFile
Machine *-- CPU
Machine *-- Screen

CPU *-- Instruction : ir
CPU ..> Memory : fetch
CPU ..> RegisterFile
```

CPU ..> CommandFactory : uses

CPU ..> ExecutionContext : builds

CommandFactory ..> Command : creates

Command <|-- LoadFromMemory

Command <|-- LoadImmediate

Command <|-- StoreToMemory

Command <|-- MoveRegister

Command <|-- AddTwosComplement

Command <|-- AddFloat

Command <|-- Jump

Command <|-- Halt

Command ..> Instruction : reads

Command ..> ExecutionContext : uses

ExecutionContext o-- Memory

ExecutionContext o-- RegisterFile

ExecutionContext o-- Screen

@enduml

why you did the design this way??

I made the design choices for a few main reasons, and some of them may be more than your project needs.

1. Each class has one job (Single Responsibility)

- `Memory` and `RegisterFile` only store data.

- `Instruction` only decodes the 4 hex digits.

- `CPU` only runs the fetch-decode-execute cycle.

- `Machine` only wires everything together and exposes `step()`/`run()`.

This matches your original idea, so I kept the skeleton and tightened it.

2. Command pattern instead of a big `switch`

Without it, `CPU::execute()` becomes a 10+ case switch that you edit every time you add an instruction. With it, each opcode is its own small class, so adding one means adding a class rather than touching the CPU. This is the Open/Closed principle, and it's also what most OOP courses want to see.

3. `ExecutionContext`

Commands need memory, registers, PC, the halt flag, and the screen. Passing five parameters to every `execute()` is messy, so I bundled them into one struct of references.

4. Composition vs. dependency

- `Machine` owns Memory, Registers, CPU, and Screen (composition), because they live and die with the machine.

- `CPU` only uses Memory and Registers (dependency), because it doesn't own them. This lets you test the CPU alone with fake memory.

5. `Screen` as a separate class

Opcode 3 with address `00` writes to the screen. Keeping I/O out of `Memory` stops it from depending on the console.

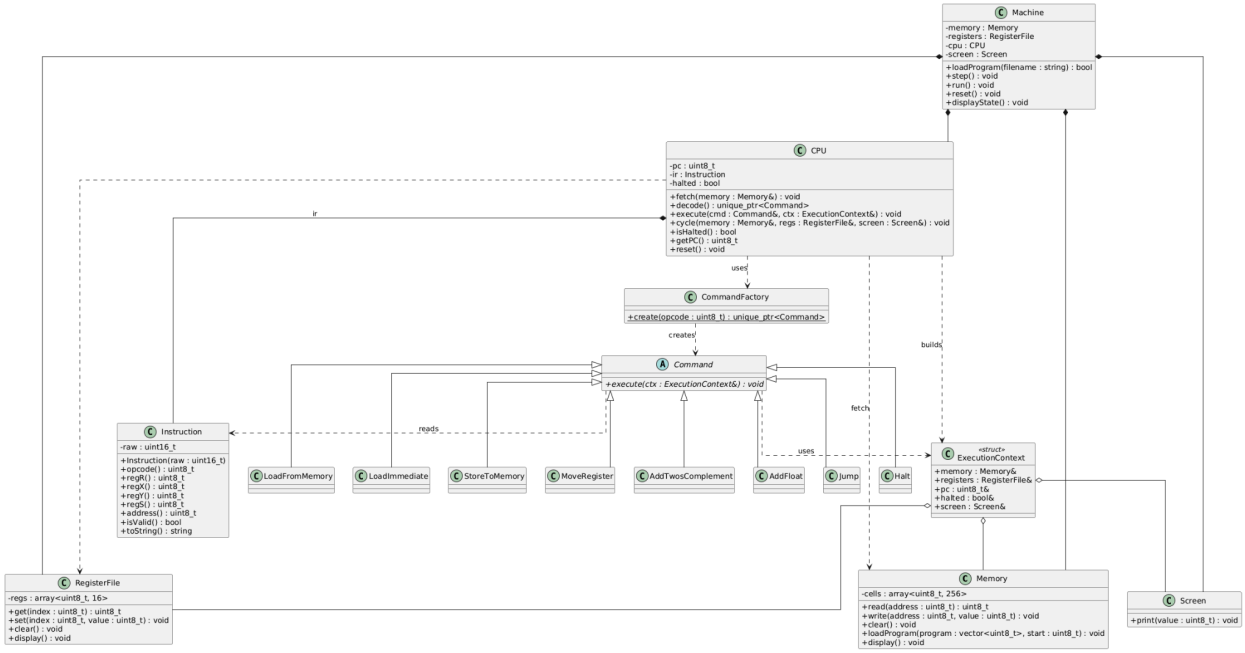
Honest trade-off: for a university assignment, the Command + Factory + Context part is probably over-engineered. The simpler version is `Memory`, `RegisterFile`, `Instruction`, `CPU` (with a `switch`), `Machine`, and optionally `Screen`. That is easier to write, debug, and explain, and it's usually enough to satisfy the OOP requirements. I'd only keep the Command design if your instructor values design patterns

or you plan to extend the instruction set.

One correction: in my notes I said the float instruction is opcode 5. In the Vole instruction set, opcode **5** is add (two's complement) and opcode **6** is add (floating point).

If you want, I can give you the simplified diagram instead.

The first plantuml:



Chat link : https://chatgpt.com/share/6ade014f-37d0-83ea-9931-48874b64821c

2nd AI PROMPTS:

ودايرين تصميم كلاسات ممتاز ومختلف من وجهة نظرك c++ بـ vole machine بنصمم محاكي لـ.

فيه عداد البرنامج cpu و معالج rf، لـ r0 من register و 16 مسجل ff، فيها 256 مكان من 00 لـ main memory السيستم فيه ذاكرة run أو step by step و كلاس رئيسي للمحاكي بيمشي الأوامر إما ir، ومسجل التعليمة pc

للرسمة مع توضيح بسيط plantuml اقترح لي أفضل هيكلية كلاسات من وجهة نظرك بدون ما تلتزم بالتصاميم التقليدية، واكتب لي كود للعلاقات بين الكلاسات

REPLIES:

بهيكلية قوية ومختلفة، ما أنصحكم تخلوا كل المسؤوليات في C++ بـ Vole Machine أنا من وجهة نظري، لو دايرين تعملوا محاكي Instruction Execution، وتنفيذ التعليمات Hardware، الأفضل نفصل بين العتاد. `Simulator` أو `CPU` كلاس واحد اسمه Simulation Control، بحيث كل كلاس يكون عنده مسؤولية واضحة والتحكم في المحاكاة.

الفكرة المميزة هنا: نخلي الـ CPU مسؤول عن تنفيذ دورة التعليمة، والـ Simulator مسؤول عن تحديد هل التنفيذ حيكون `Step` أو `Run`، والـ Memory والـ Registers يكونوا مكونات مستقلة يمكن اختبارها وتطويرها بسهولة.

1. الهيكلية المقترحة للكلاسات

VoleSimulator

واجهة التحكم بالمحاكاة

step() · run() · reset() · halt

يتحكم في التنفيذ

CPU

Fetch → Decode → Execute

PC · IR · تنفيذ التعليمات

MainMemory

بايت/موقع ذاكرة 256

00–FF

RegisterFile

مسجلًا 16

R0–RF

InstructionSet / Decoder

ويحدد العملية المطلوبة Opcode يفسر

الرسمة دي توضيحية للمفهوم؛ في التصميم الفعلي حنضيف كلاس مستقل للتعليمات، وكلاس لحالة التنفيذ، عشان التصميم يكون قابل للتوسع بدل ما نربط كل شيء ببعض مباشرة.

2. الكلاسات المقترحة بالتفصيل

أنا بقترح 7 كلاسات أساسية، مع إمكانية إضافة واجهة رسومية لاحقًا بدون ما نغيّر منطق المحاكي.

الكلاس	مسؤوليته
`VoleSimulator`	تشغيل المحاكي، تنفيذ خطوة واحدة أو تشغيل البرنامج كاملًا، وإيقافه.
`CPU`	إدارة دورة التعليمة، وتحديث `PC`و `IR`، وتنفيذ العمليات.

| `MainMemory` | تخزين البيانات والتعليمات في 256 موقعًا. |

| `RegisterFile` | إدارة المسجلات الستة عشر من `R0` إلى `RF`. |

| `InstructionDecoder` | تحليل التعليمة واستخراج الـ Opcode والـ Operands. |

| `Instruction` | تمثيل التعليمة بعد فك ترميزها. |

| `MachineState` | تمثيل حالة المحاكي: جاهز، يعمل، متوقف، أو انتهى بسبب خطأ. |

نقطة التصميم المهمة: ما نخلي `InstructionDecoder` ينفّذ التعليمة بنفسه؛ وظيفته الفهم والتحليل فقط، بينما التنفيذ يكون داخل `CPU`. الفصل دا بيسهّل اكتشاف الأخطاء وإضافة تعليمات جديدة.

4. شرح العلاقات بين الكلاسات

1\. Composition — المحاكي يمتلك مكونات الجهاز

العلاقة بين `VoleSimulator` وكل من `CPU` و`MainMemory` و`RegisterFile` هي Composition، الممثلة بالماسة المملوءة `*--`. المحاكي ينشئ هذه المكونات ويدير دورة حياتها.

2\. CPU يعتمد على الذاكرة والمسجلات

الـ CPU يقرأ التعليمات والبيانات من `MainMemory`، ويقرأ ويكتب قيم المسجلات عبر `RegisterFile`. لكنه لا يحتاج إلى امتلاكهما داخله؛ المحاكي يمررهما له.

3\. Decoder ينشئ Instruction

الـ CPU للـ `Instruction` عن المعاملات. بعد ذلك يرجع كائن Opcode يأخذ التعليمة الخام، ويفصل الـ `InstructionDecoder` الـ كي ينفّذ العملية المناسبة.

4\. MachineState يحدد حالة التشغيل

يحتفظ بحالة المحاكي، وبيستخدمها لمنع التشغيل إذا انتهى البرنامج أو حدث خطأ، بدل الاعتماد على متغيرات `VoleSimulator` الـ متفرقة في عدة كلاسات.

5. كيف نربط التصميم بعمليتي Step وRun؟

الجميل في التصميم دا إننا ما محتاجين نكتب منطق تنفيذ مختلف لكل وضع.

- `step()` ينفّذ تعليمة واحدة فقط، ثم يرجع التحكم للمستخدم.

- `run()` يكرر تنفيذ الخطوات حتى يصل البرنامج إلى تعليمة التوقف أو تحدث حالة خطأ.

- `reset()` يعيد الـ PC والمسجلات وحالة المحاكي إلى القيم الابتدائية، مع تحديد هل نريد مسح الذاكرة أيضًا أم الاحتفاظ بالبرنامج المحمّل.

بالتالي، `run()` داخليًا يستفيد من نفس منطق تنفيذ الخطوة، وما بنكرر كود الـ Fetch والـ Decode والـ Execute مرتين.

6. ملاحظة مهمة قبل اعتماد الرسم

بالتالي، 8 bits عنوانه PC وأن الـ 8 bits يخزن IR 16 وأن كل مسجل والـ bits في الرسم افترضت أن كل موقع ذاكرة يخزن 8 bits، Vole وفقًا لذلك. لازم تتأكدوا من مواصفات نسخة PC التعليمة ذات 16 بت تحتاج عادةً إلى قراءتين من الذاكرة، مع تحديث الـ Machine المطلوبة في المقرر قبل تثبيت التفاصيل دي.

رأيي النهائي: ابدأوا بالـ 7 كلاسات دي. هيكلية واضحة، قابلة للاختبار والتوسع، ومش معقدة زيادة عن المطلوب لمشروع جامعي. ولو بحيث يصبح المعالج `CPU` مستقلًا عن `InstructionExecutor` عايزين تميّزوا التصميم أكثر في مرحلة لاحقة، ممكن تضيفوا Opcode مسؤولًا عن دورة التعليمة، بينما التنفيذ الفعلي لكل يكون في مكوّن متخصص.

PlantUML CODE:

@startuml

```
title Vole Machine Simulator - Class Diagram

skinparam classAttributeIconSize 0
skinparam linetype ortho
hide empty members

class VoleSimulator {
    - cpu : CPU
    - memory : MainMemory
    - registers : RegisterFile
    - state : MachineState
    + step() : void
    + run() : void
    + reset() : void
    + stop() : void
}

class CPU {
    - pc : uint8_t
    - ir : uint16_t
    - halted : bool
    + fetch() : uint16_t
    + executeStep() : void
```

```
    + reset() : void
    + getPC() : uint8_t
    + getIR() : uint16_t
}

class MainMemory {
    - cells : uint8_t[256]
    + read(address : uint8_t) : uint8_t
    + write(address : uint8_t, value : uint8_t) : void
    + loadProgram() : void
    + clear() : void
}

class RegisterFile {
    - registers : uint16_t[16]
    + read(index : uint8_t) : uint16_t
    + write(index : uint8_t, value : uint16_t) : void
    + clear() : void
}

class InstructionDecoder {
    + decode(raw : uint16_t) : Instruction
}
```

```
class Instruction {
    + opcode : uint8_t
    + operand1 : uint8_t
    + operand2 : uint8_t
    + operand3 : uint8_t
}

enum MachineState {
    Ready
    Running
    Halted
    Error
}

VoleSimulator *-- CPU
VoleSimulator *-- MainMemory
VoleSimulator *-- RegisterFile
VoleSimulator --> MachineState

CPU --> MainMemory : fetch / read / write
CPU --> RegisterFile : access registers
CPU --> InstructionDecoder : decode
InstructionDecoder ..> Instruction : creates
```

note right of MainMemory

256 addresses:

00 through FF

Each cell stores 8 bits.

end note

note right of RegisterFile

16 registers:

R0 through RF

Each register stores 16 bits.

end note

note right of CPU

PC: 8-bit address

IR: 16-bit instruction

end note

@enduml

and why did you choose this one exactly??

I chose this design because it balances simplicity, flexibility, and good object-oriented programming principles. The goal is not just to make the simulator work, but to make the code easy to understand, test, debug, and extend.

Here are the main reasons behind my choices.

1. Why did I separate the classes?

Class	Why I chose it
`VoleSimulator`	It controls the whole simulation and provides `step()` and `run()` without handling low-level hardware operations itself.
`CPU`	It handles the instruction cycle: Fetch, Decode, and Execute. This keeps the processor logic in one place.
`MainMemory`	It manages the 256 memory locations independently, so memory operations don't get mixed with CPU logic.
`RegisterFile`	It manages all 16 registers in one class instead of creating 16 separate objects or variables.
`InstructionDecoder`	It separates instruction decoding from execution, making the CPU easier to maintain.
`Instruction`	It represents a decoded instruction in a structured way instead of passing raw instruction values everywhere.
`MachineState`	It clearly tracks whether the simulator is ready, running, halted, or in an error state.

2. Why not put everything inside the CPU?

Because the CPU would become responsible for too many things: storing memory, managing registers, decoding instructions, and controlling the simulation.

That creates tight coupling: changing one part could affect several other parts.

With the proposed design, each class has one clear responsibility. This follows the Single Responsibility Principle (SRP) and makes testing individual components easier.

3. Why did I choose composition?

The simulator has a CPU, memory, and register file.

That is a natural has-a relationship, so composition is more appropriate than inheritance here.

For example:

- `VoleSimulator` owns the `CPU`.
- `VoleSimulator` owns the `MainMemory`.
- `VoleSimulator` owns the `RegisterFile`.

We don't need inheritance between these classes because a CPU is not a type of memory, and memory is not a type of CPU.

4. Why separate `step()` and `run()`?

Both operations should use the same instruction-execution logic.

- `step()` executes one instruction.
- `run()` repeatedly executes instructions until the program halts or an error occurs.

This avoids duplicating the Fetch–Decode–Execute logic and reduces the chance of inconsistent behavior

between the two modes.

5. What makes this design different?

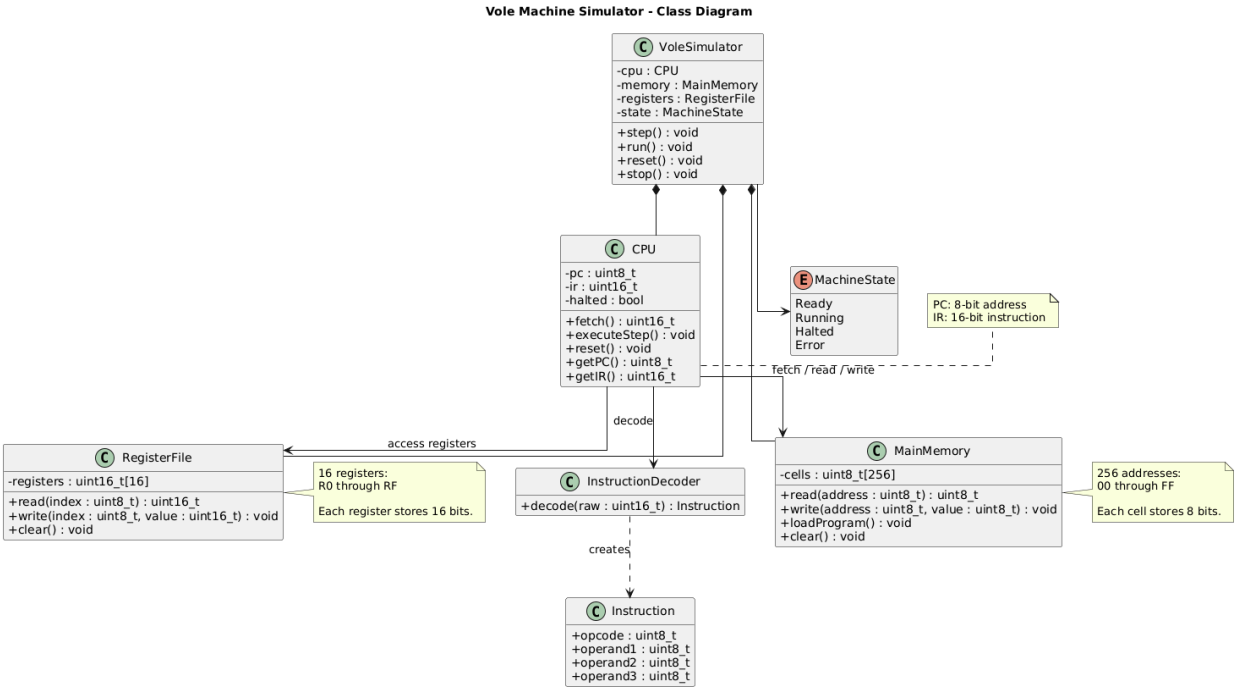
The main design decision is separating simulation control from hardware behavior.

A more traditional design might put nearly everything inside `CPU` and let the main program control it directly. My design gives each component a defined role, while keeping the overall architecture manageable for a C++ university project.

I would also consider one improvement: adding an `InstructionExecutor` class if the instruction set becomes large. It could handle individual operations, while the CPU manages the instruction cycle. I wouldn't add it immediately unless the project requirements justify the extra complexity.

In short: I chose this architecture because it is modular without being unnecessarily complicated. It supports the required functionality now and leaves room for future features without requiring a complete redesign.

The second plantuml:



Chat link:
https://gemini.google.com/share/d/1P2P2lfpQtui6x8OM5s0Js8rRtcRa_Rxy?usp=drive_link

3rd AI PROMPTS:

ودايرين تصميم كلاسات ممتازة ومختلفة، مع التركيز الشديد على فصل c++ بـ vole machine شغالين على تصميم محاكي لـ المسؤوليات وسهولة الصيانة والاختبار.

جواه cpu و معالج rf، لـ r0 من register و 16 مسجل ff، فيها 256 مكان من 00 لحد main memory السيستم بيحتوي على ذاكرة run. أو step by step بالإضافة لكلاس رئيسي للمحاكي بيشغل الأوامر إما ir، ومسجل التعليمة pc عداد البرنامج

للرسمة مع شرح plantuml اكتب لي هيكلية كلاسات واضحة واقترح أسلوب ممتاز لتنظيم الكلاسات والعلاقات بينها، واكتب لي كود بسيط لسبب اختيارك للعلاقات دي

بشكل عملي، **SOLID** يتطلب توظيف مبادئ (Vole Machine / Brookshear Machine) تصميم معمارية محاكي آلة فول (Interface Segregation). وفصل الواجهات (Single Responsibility Principle) وتحديداً فصل المسؤوليات

إلى كلاس مستقل، مع إدخال كلاسات مساعدة للفك والترميز (Hardware Component) في هذا التصميم، تم فصل كل مكوّن عتادي (Decoding/Execution) وحالة الجهاز، لتسهيل **الاختبار (Unit Testing) **و**الصيانة والتطوير**.

---

1. الهيكلية المقترحة للتصميم (Class Architecture)

المكونات الأساسية والعلاقات:

1. **`Memory` (الذاكرة الرئيسية):**

* **المسؤولية:** احتواء 256 بايت (من `00` إلى `FF`).

* **الميزات:** يوفر واجهة بسيطة لقراءة وكتابة بايت منفرد (`readByte`, `writeByte`).

2. **`RegisterFile` (سجل المسجلات):**

* **المسؤولية:** إدارة 16 مسجل عام (من `R0` إلى `RF`).

* **الميزات:** قراءة وكتابة قيمة المسجل (`getRegister`, `setRegister`).

3. **`Instruction` (كلاس التعليمة):**

* **المسؤولية:** تمثيل التعليمة الواحدة (16-bit أو 2-bytes) المكونة من Opcode (4 bits) وOperands/Registers (12 bits).

* **الفائدة:** يسهل عملية الفحص والتحليل بدون الحاجة للتعامل مع العمليات الثنائية (Bitwise Operations) في كل مكان داخل الكود.

4. **`Decoder` (فك الترميز):**

* **المسؤولية:** تحويل البايتات الموجودة في الذاكرة إلى كائن `Instruction` نقي.

5. **`ALU` (وحدة الحساب والمنطق):**

* **المسؤولية:** تنفيذ العمليات الحسابية والمنطقية (جمع، دمج منطقي OR/AND، تدوير Bitwise Rotate، إلخ) بتنسيقات (Two's complement و Floating Point الخفيفة الخاصة بآلة فول).

* **الفائدة:** فصل منطق الحساب تماماً عن الـCPU ومسجلات التحكم.

6. **`CPU` (وحدة المعالجة المركزية):**

* **المسؤولية:** إدارة الدورة الأساسية (Fetch-Decode-Execute Cycle).

* **المكونات الداخلية:** يحتوي على `PC` (Program Counter) و `IR` (Instruction Register)، ويتعامل مع `ALU` و `Decoder`.

7. **`VoleSimulator` (كلاس المحاكي الرئيسي / Controller):**

* **المسؤولية:** واجهة التحكم العالية (High-Level Controller) التي تربط بين الـ `CPU` والـ `Memory`.

* **الميزات:** يوفر دوال التشغيل `step()` و `run()` وحظر التوقف `halt()`.

8. **`IUI / ConsoleUI` (واجهة المستخدم):**

* **المسؤولية:** عرض حالة المحاكي واستقبال الأوامر من المستخدم دون التداخل مع اللوجيك الخاص بالمحاكي.

3. سبب اختيار هذه العلاقات وطرق التنظيم

1. **الاعتماد بواسطة المراجع (Dependency Injection / References):**

* كلاس `CPU` يحمل مرجعاً (`RegisterFile&`) بإنشاء هذا يسمح لاحقاً **Mock RegisterFile** ولا يمتلك المسجلات بنفسه.

بمفرده دون الحاجة لتشغيل كامل المحاكي CPU واختبار الـ.

2. **علاقة التركيب القوية (Composition `*--`):**

* كلاس `VoleSimulator` هو الحاوية الرئيسية (Facade Controller). يمتلك الـ `Memory` والـ `RegisterFile` والـ `CPU` في مكان واحد وتجنب تسريب الذاكرة (Memory Leaks) لضمان إدارة دورة حياتهم (Lifecycle Management).

3. **فصل Decoder و ALU (Single Responsibility):**

* جعل `Decoder` كـ Utility class يسعف في اختبار تحويل التعليمة بشكل منفصل (Unit Test)، حيث يدخل له `0x34 0x12` فيرجّع كائن `Instruction` بـ `opcode = 1`, `r = 2`, `address = 0x34`.

* فصل `ALU` سهلاً ومستقلاً تماماً عن مسارات Vole الخاصة بـ **Floating Point Add** يجعل اختبار العمليات المعقدة مثل الـ CPU والـ Memory.

4. **استخدام حالات المعالج (State Machine - `CpuState`):**

* استخدام `CpuState` يسهل للواجهة البرمجية (CLI أو GUI) معرفة إن كان البرنامج متوقفاً برمز Halted (التعليمة `C000`) أو نظيفاً جداً `run()` و `step()` يحتاج تدخل، مما يجعل تنفيذ دالة

PlantUML CODE:

```
@startuml VoleMachineArchitecture


skinparam classAttributeIconSize 0

skinparam backgroundColor #FFFFFF

skinparam class {

   BackgroundColor #F8F9FA
```

```
    BorderColor #343A40
    ArrowColor #495057
}

package "Core Components" {
    class Memory {
        - data: std::array<uint8_t, 256>
        + readByte(address: uint8_t): uint8_t
        + writeByte(address: uint8_t, value: uint8_t): void
        + clear(): void
    }

    class RegisterFile {
        - registers: std::array<uint8_t, 16>
        + get(regNum: uint8_t): uint8_t
        + set(regNum: uint8_t, value: uint8_t): void
        + clear(): void
    }

    struct Instruction {
        + opcode: uint8_t
        + r: uint8_t
        + x: uint8_t
        + y: uint8_t
```

```
        + address: uint8_t
    }

    class Decoder {
        + static decode(byte1: uint8_t, byte2: uint8_t): Instruction
    }

    class ALU {
        + addTwoComplement(val1: uint8_t, val2: uint8_t): uint8_t
        + addFloatingPoint(val1: uint8_t, val2: uint8_t): uint8_t
        + bitwiseOR(val1: uint8_t, val2: uint8_t): uint8_t
        + bitwiseAND(val1: uint8_t, val2: uint8_t): uint8_t
        + bitwiseXOR(val1: uint8_t, val2: uint8_t): uint8_t
        + rotateRight(val: uint8_t, steps: uint8_t): uint8_t
    }
}

package "Execution Engine" {
    enum CpuState {
        READY
        RUNNING
        HALTED
        ERROR
    }
```

```
class CPU {
    - pc: uint8_t
    - ir: uint16_t
    - state: CpuState
    - alu: ALU
    - registers: RegisterFile&
    + CPU(regs: RegisterFile&)
    + fetch(mem: Memory&): void
    + executeNext(mem: Memory&): void
    + getPC(): uint8_t
    + getIR(): uint16_t
    + getState(): CpuState
    + reset(): void
}

class VoleSimulator {
    - memory: Memory
    - registers: RegisterFile
    - cpu: CPU
    + loadProgram(startAddress: uint8_t, bytes: vector<uint8_t>): bool
    + step(): bool
    + run(): void
    + reset(): void
```

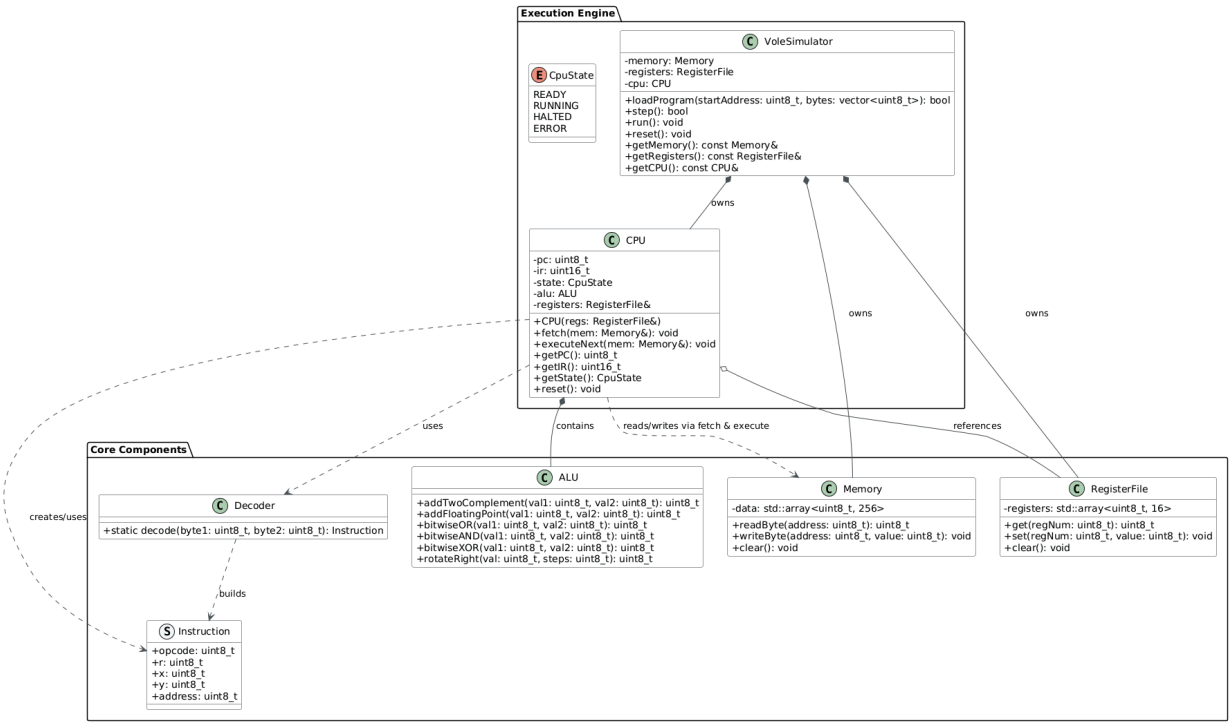
```
        + getMemory(): const Memory&
        + getRegisters(): const RegisterFile&
        + getCPU(): const CPU&
    }
}

' Relations and Dependencies
VoleSimulator *-- Memory : owns
VoleSimulator *-- RegisterFile : owns
VoleSimulator *-- CPU : owns

CPU o-- RegisterFile : references
CPU *-- ALU : contains
CPU ..> Decoder : uses
CPU ..> Instruction : creates/uses
CPU ..> Memory : reads/writes via fetch & execute

Decoder ..> Instruction : builds
@endif
@enduml
```

The theard plantuml:



Design Selection & Refinement Rationale :

Based on our AI comparison, we selected Gemini's design (Design #3) as our baseline architecture because it effectively separated the core domain components from the execution engine. To finalize our model, we refined Design #3 by adding a memory-mapped Screen class for address 0xFF handling, along with isValid() and toString() helper methods in the Instruction component.

Comparison Table:

AI Tool	Key Strengths	Key Weaknesses	PlantUML Output Quality
ChatGPT	Applied advanced OOP patterns (Command & Factory) for Open/Closed principle; included a dedicated Screen class for I/O handling.	Over-engineered for a student project; introduced unnecessary class complexity for basic instructions.	Very High (Complete syntax with full attribute types and method signatures)

AI Tool	Key Strengths	Key Weaknesses	PlantUML Output Quality
Claude	Clear separation between simulation control and hardware; introduced MachineState enum for runtime status tracking	Missing a dedicated Screen class for address FF output; combined register specs without full bit-width distinction.	High (Clean UML layout with useful diagram notes and standard relationships).
Gemini	Excellent architectural packaging (Execution Engine vs. Core Components); dedicated ALU class for arithmetic testing.	Initial draft lacked explicit memory-mapped Screen logic and instruction validation helpers.	Very High (Professional package layout, colored styling, and precise relationship arrows)

4. Final Design :

Chat link:https://gemini.google.com/share/d/1nnBOwaL9PSFjdF2sEiCYRt2oQT_24lVo?usp=drive_link

PlantUML Source Code:

```
@startuml

skinparam classAttributeIconSize 0



package "Execution Engine" {

 enum CpuState {

  READY

  RUNNING

  HALTED
```

```
  ERROR
}

class VoleSimulator {
  - memory: Memory
  - registers: RegisterFile
  - cpu: CPU
  - screen: Screen
  + loadProgram(startAddress: uint8_t, bytes: vector<uint8_t>): bool
  + step(): bool
  + run(): void
  + reset(): void
  + getMemory(): const Memory&
  + getRegisters(): const RegisterFile&
  + getCPU(): const CPU&
  + getScreen(): const Screen&
}

class CPU {
```

```
    - pc: uint8_t
    - ir: uint16_t
    - state: CpuState
    - alu: ALU
    - registers: RegisterFile&
    + CPU(regs: RegisterFile&)
    + fetch(mem: Memory&): void
    + executeNext(mem: Memory&): void
    + getPC(): uint8_t
    + getIR(): uint16_t
    + getState(): CpuState
    + reset(): void
  }
}

package "Core Components" {
  class Memory {
    - data: std::array<uint8_t, 256>
    + readByte(address: uint8_t): uint8_t
```

```
  + writeByte(address: uint8_t, value: uint8_t): void
  + clear(): void
}

class RegisterFile {
  - registers: std::array<uint8_t, 16>
  + get(regNum: uint8_t): uint8_t
  + set(regNum: uint8_t, value: uint8_t): void
  + clear(): void
}

class ALU {
  + addTwoComplement(val1: uint8_t, val2: uint8_t): uint8_t
  + addFloatingPoint(val1: uint8_t, val2: uint8_t): uint8_t
  + bitwiseOR(val1: uint8_t, val2: uint8_t): uint8_t
  + bitwiseAND(val1: uint8_t, val2: uint8_t): uint8_t
  + bitwiseXOR(val1: uint8_t, val2: uint8_t): uint8_t
  + rotateRight(val1: uint8_t, steps: uint8_t): uint8_t
}
```

```
class Decoder {
  + {static} decode(byte1: uint8_t, byte2: uint8_t): Instruction
}

struct Instruction {
  + opcode: uint8_t
  + r: uint8_t
  + x: uint8_t
  + y: uint8_t
  + address: uint8_t
  + isValid(): bool
  + toString(): std::string
}

class Screen {
  - outputBuffer: std::string
  + displayByte(value: uint8_t): void
  + clear(): void
```

```
    + getOutput(): std::string

  }

}


' Relationships from Execution Engine

VoleSimulator *-- Memory : owns

VoleSimulator *-- RegisterFile : owns

VoleSimulator *-- CPU : owns

VoleSimulator *-- Screen : owns


CPU o-- RegisterFile : references

CPU *-- ALU : contains

CPU ..> Decoder : uses

CPU ..> Memory : reads/writes via fetch & execute

CPU ..> Instruction : creates/uses

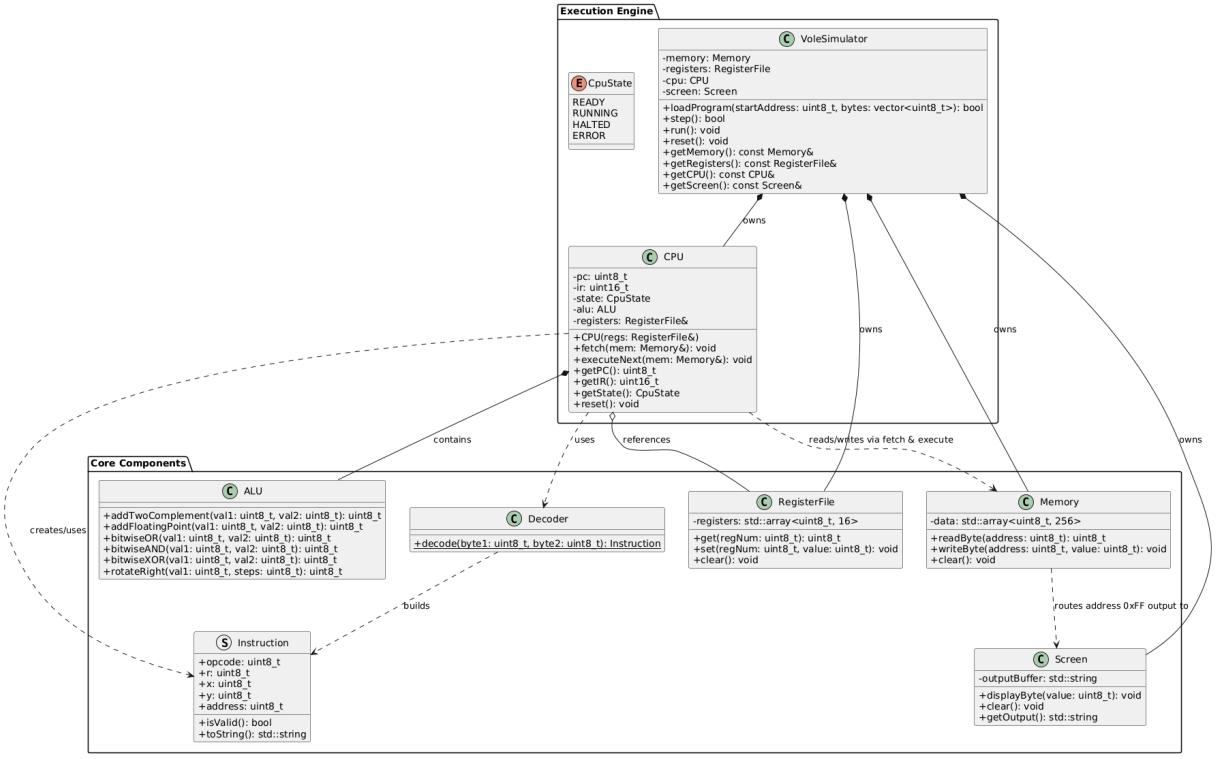

' Core Components Relationships

Decoder ..> Instruction : builds

Memory ..> Screen : routes address 0xFF output to
```

@enduml

Diagram Image:



5. Examples (Custom Vole Assembly Programs) :

Program 1: Three-Number Addition

The Code :

```
2105
2206
2307
5412
5543
35FF
C000
```

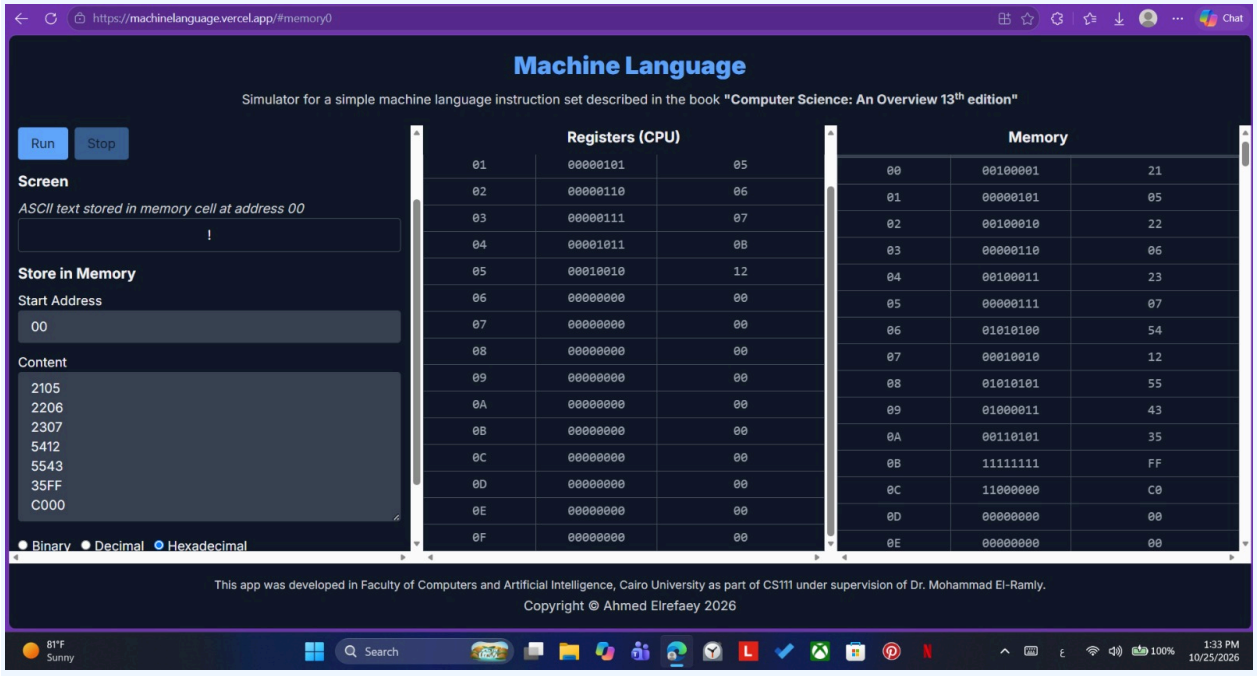
Step-by-Step Trace Table:

Address	Instruction	Opcode Action	Register / Memory State After Execution
00 - 01	2105	Load Immediate	R1 = 0x05
02 - 03	2206	Load Immediate	R2 = 0x06
04 - 05	2307	Load Immediate	R3 = 0x07
06 - 07	5412	Add (R1 + R2)	R4 = 0x0B
08 - 09	5543	Add (R4 + R3)	R5 = 0x12
0A - 0B	35FF	Store Memory	Memory[0xFF] = 0x12
0C - 0D	C000	Halt	Execution Stopped

Code Explanation:

In this program we add three numbers together and print the final result on the screen. First, instructions 2105, 2206, and 2307 load the values 0x05, 0x06, and 0x07 into registers R1 R2 and R3. Then 5412 adds R1 and R2 together and saves the result (0x0B) in R4. Next, 5543 adds R4 and R3 together and stores the final sum (0x12) in R5. After that 35FF writes the result from R5 to memory address FF to display it on the screen and finally C000 stops the machine.

Simulator Screenshot:



Program 2: Register Value Swap (SWAP Operation)

The Code:

```
210A
220B
4013
4021
4032
31FF
C000
```

Step-by-Step Trace Table:

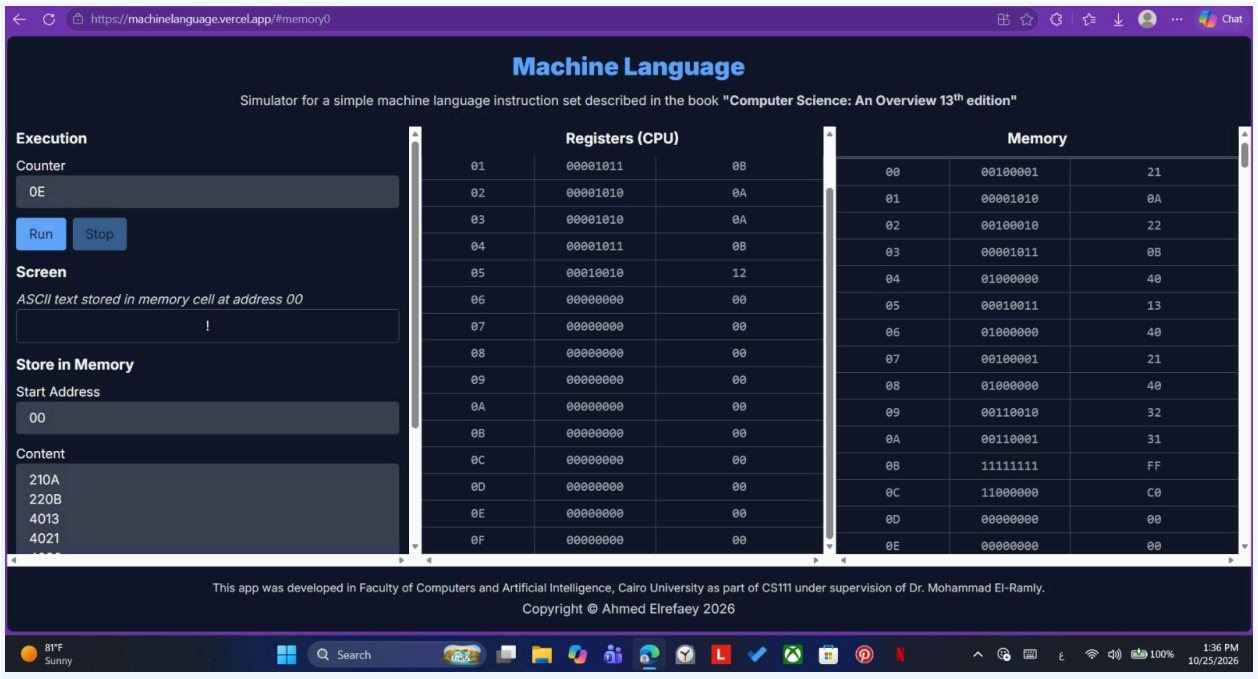
Address	Instruction	Opcode Action	Register State After Execution
00 - 01	210A	Load Immediate	R1 = 0x0A
02 - 03	220B	Load Immediate	R2 = 0x0B
04 - 05	4013	Move Register (R1 to R3)	R3 = 0x0A (Temp)
06 - 07	4021	Move Register (R2to	R1 = 0x0B (Swapped)

Address	Instruction	Opcode Action	Register State After Execution
		R1)	
08 - 09	4032	Move Register (R3to R2)	R2 = 0x0A (Swapped)
0A - 0B	31FF	Store Memory	Memory[0xFF] = 0x0B
0C - 0D	C000	Halt	Execution Stopped

Code Explanation:

In this program, we want to swap two numbers between Register 1 and Register 2 using Register 3 as a temporary holder. First 210A and 220B load the values 0x0A and 0x0B into R1 and R2. Then 4013 saves the value of R1 into R3 so we don't lose it. Next 4021 moves the value from R2 into R1 and 4032 puts the saved value from R3 into R2. Finally 31FF sends the new R1 value to address FF to show it on screen and C000 stops the machine.

Simulator Screenshot:



6. Submission Links:

- **Presentation Video Link:**
  https://drive.google.com/file/d/16K1h1PGa79EUbMnIs6K09A5yaPay6ouB/view?usp=drivesdk
- **Team GitHub Repository:**
  https://github.com/mozannageeb20253016/Vole-Machine-Simulator.git